Zurich University of Applied Sciences

School of Management and Law

Prototyping – Vorlesung 5

SvelteKit Teil 3: Bindings, Komponenten, Routing



Building Competence. Crossing Borders.

Max Meisterhans

max.meisterhans@zhaw.ch

Lernziele

- Ihr wisst, wie ihr Bootstrap in Svelte Apps verwendet.
- Ihr könnt die Navigation (Routing) in einer Svelte App implementieren.
- Ihr könnt mit Svelte Komponenten erstellen und verwenden.
- Ihr könnt mit “Props” Daten an Komponenten übergeben.



SvelteKit Projektstruktur

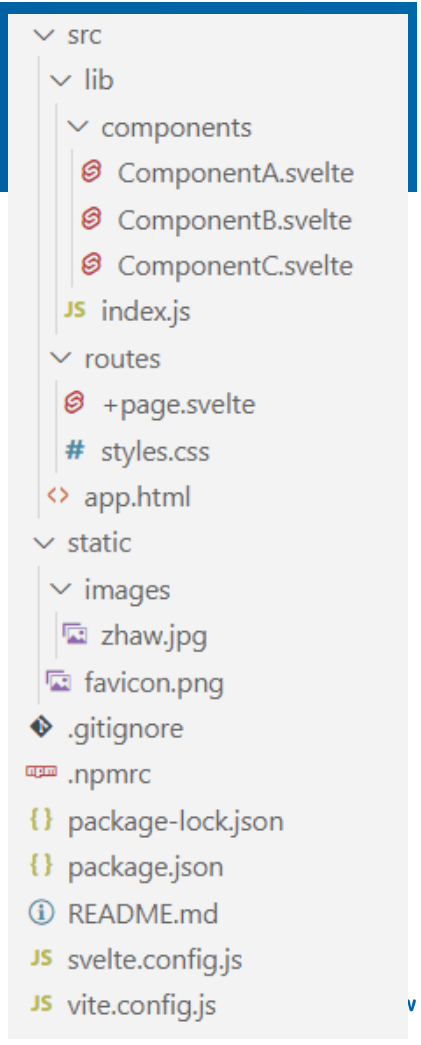
SvelteKit Projektstruktur

src: Enthält alle Programmdateien

- **lib**: Library-Dateien. Z.B. Komponenten, die auf mehreren Pages verwendet werden. Auf diese Dateien kann mit dem `#lib`-Alias zugegriffen werden, z.B. `"#lib/components/ComponentA.svelte"`
- **routes**: Enthält alle Pages (plus Layouts)

static: Enthält statische Assets (z.B. Bilder)

Siehe auch https://svelte.dev/docs/kit/project-structure



Bootstrap und CSS in SvelteKit

Bootstrap in Svelte Apps

Bootstrap CSS und JS in **src/app.html** hinzufügen:

```
<!doctype html>
<html lang="en">

<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="text-scale" content="scale" />
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet"
        integrity="sha384-T3c6CoIi6uLrA9TneNEoa7RxnatzjcDSCmG1MXxSR1GAsXEV/Dwwykc2MPK8M2HN" crossorigin="anonymous">
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"
        integrity="sha384-C6RzsynM9kWDrMNeT87bh95OGNyZPhcTNXj1NW7RuBCsyN/o0jlpcV8Qyq46cDfL"
        crossorigin="anonymous"></script>
    %sveltekit.head%
</head>

<body data-sveltekit-preload-data="hover">
    <div style="display: contents">%sveltekit.body%</div>
</body>

</html>
```

Stylesheets importieren

Stylesheets können in Svelte-Komponenten importiert werden.

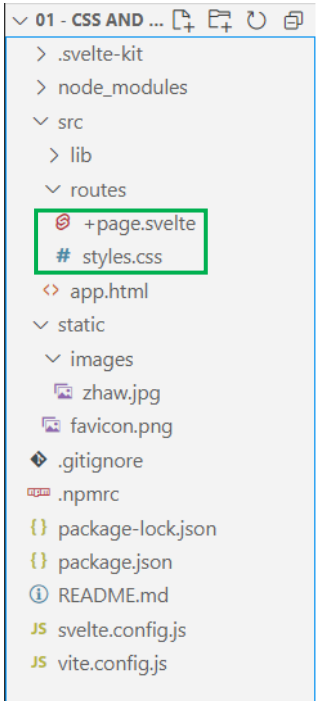
src/routes/+page.svelte

```
<script>
  import "./styles.css";
</script>


<h1>Hello World</h1>
```

src/routes/styles.css

```
h1 {
    color: orangered;
}
```



Routing

Routing

Routen sind URL-Pfade, über die auf verschiedene Komponenten zugegriffen werden kann.
In SvelteKit wird dies durch die Ordnerstruktur definiert.
Siehe auch https://svelte.dev/docs/kit/routing



Layouts

Layouts können für Teile der Website verwendet werden, die auf mehreren Pages angezeigt werden sollen. Layouts haben immer den Dateinamen *+layout.svelte* und gelten für den aktuellen Ordner und alle Unterordner. Siehe https://svelte.dev/docs/kit/routing#layout

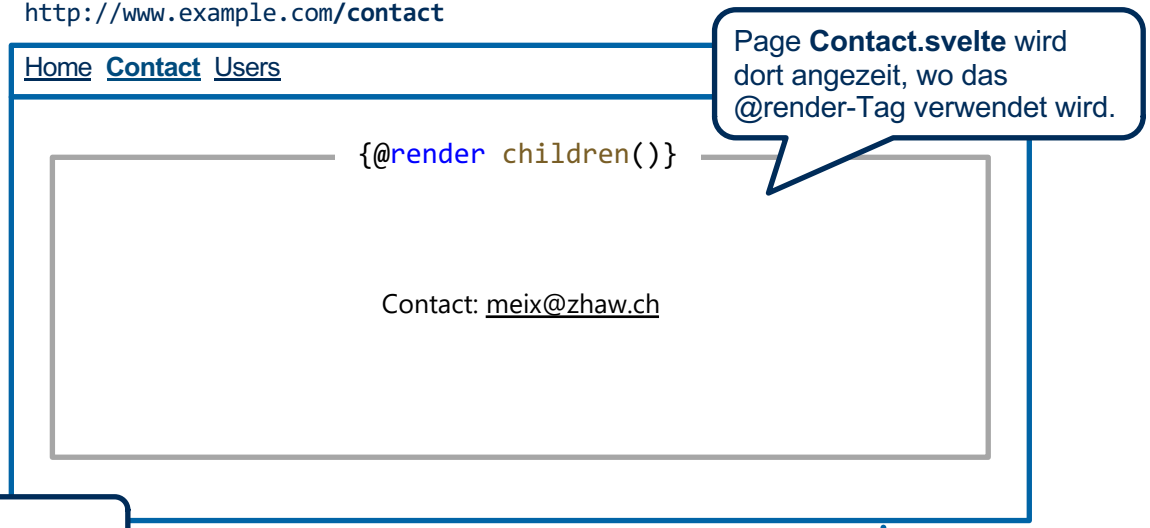
routes/+layout.svelte

```
<script>
  import './styles.css';
  let { children } = $props();
</script>

<div class="container">
  <nav>
    <a href="/">home</a>
    <a href="/contact">contact</a>
    <a href="/users">users</a>
  </nav>

  {@render children()}
</div>
```

Platzhalter für Pages.



Kurzübung – Routing und Navigation

a) Lade “Exercise_Routing_Starter" von Moodle und mache dich mit dem Code vertraut.

b) Schaue auf der nächsten Folie nach, wie die Navigation der Page funktionieren soll.

c) Installiere alle Abhängigkeiten mit `npm install`

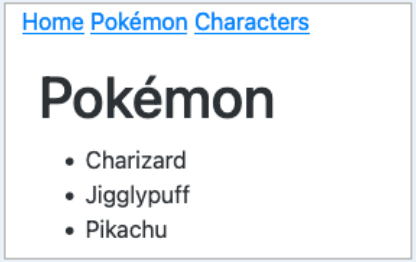
d) Starte den Development Server mit `npm run dev`

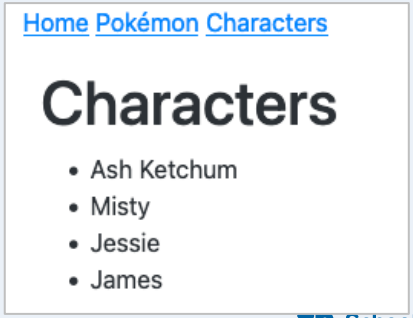
e) Erstelle Pages für *Pokémon* und *Characters*

f) Vervollständige die Navigation in +layout.svelte

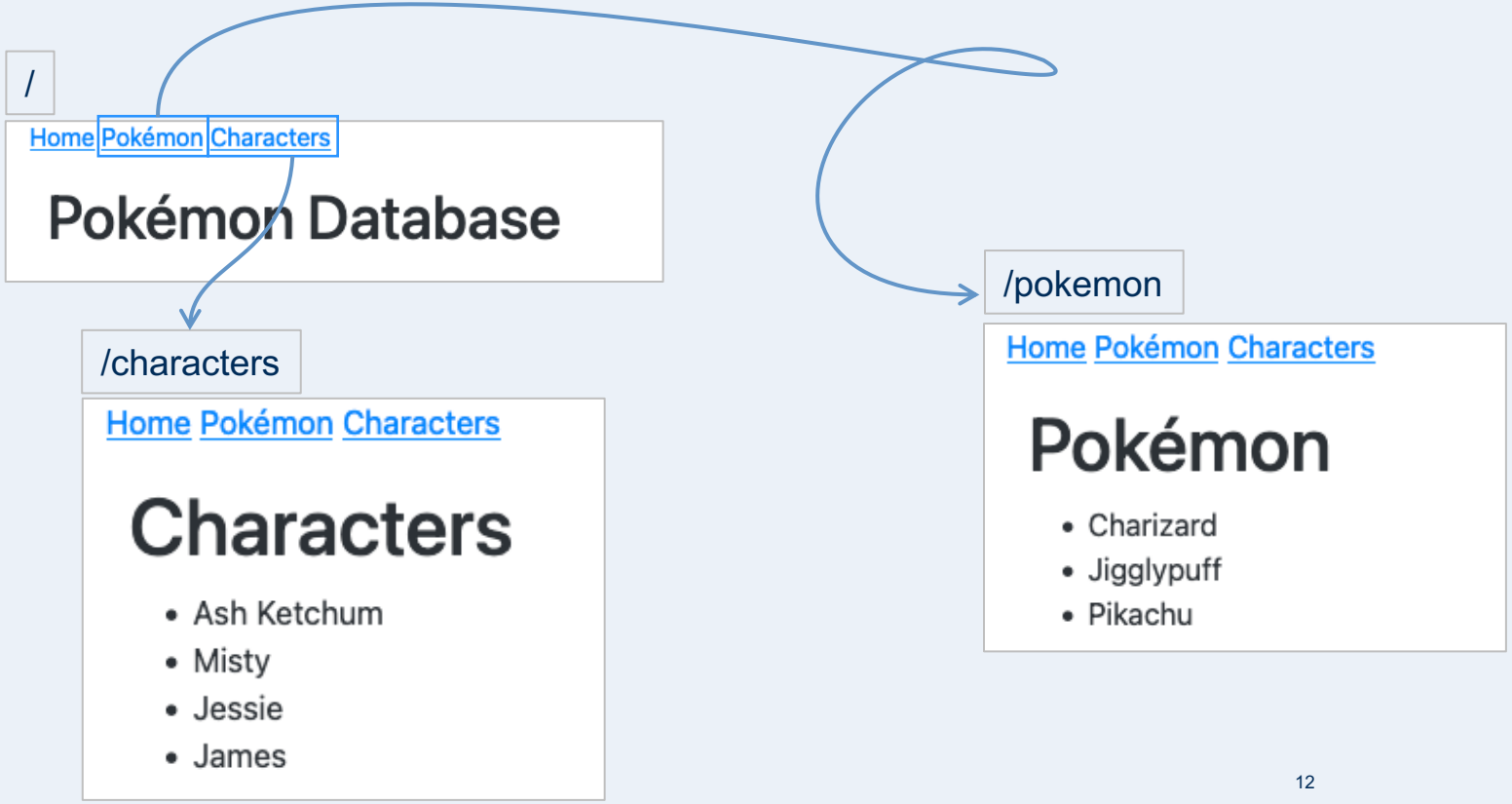
g) [Optional] Ergänze die Pages mit Bildern. Passe das Aussehen mit Styles aus einem importierten Stylesheet an (siehe Slide 7).







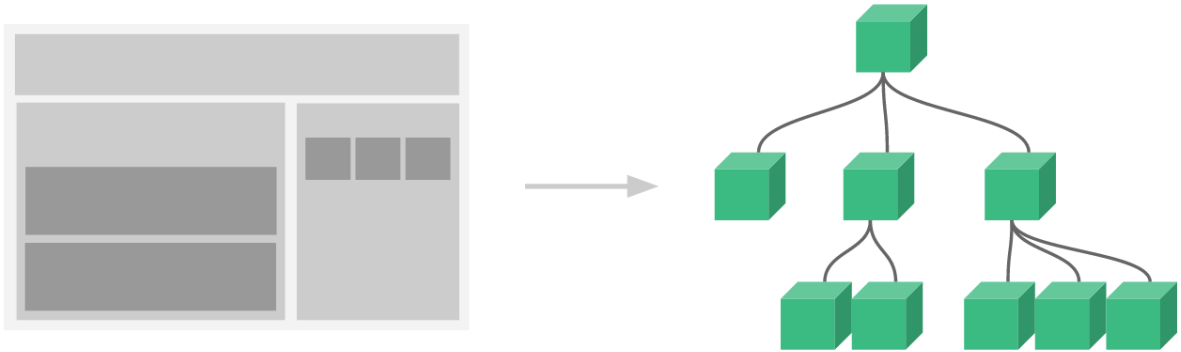
Kurzübung – Routing und Navigation



Komponenten

Svelte-Komponenten

- Komponenten sind *wiederverwendbare* Teile, um Anwendungen zu bauen.
- Komponenten können *verschachtelt* sein. (Jede Komponente kann eine beliebige Anzahl weiterer Komponenten enthalten.)
- *Beispiele* für Komponenten: Pages, Cards, Dialoge, Tooltips, Schaltflächen, Diagramme, sortierbare Tabellen, ...



Komponenten erstellen

Good practice: Komponenten im Ordner *lib/components* verwalten.

Jede Komponente wird in einer separaten Datei implementiert.

This is Component A

ComponentA.svelte

```
<p>This is Component A</p>

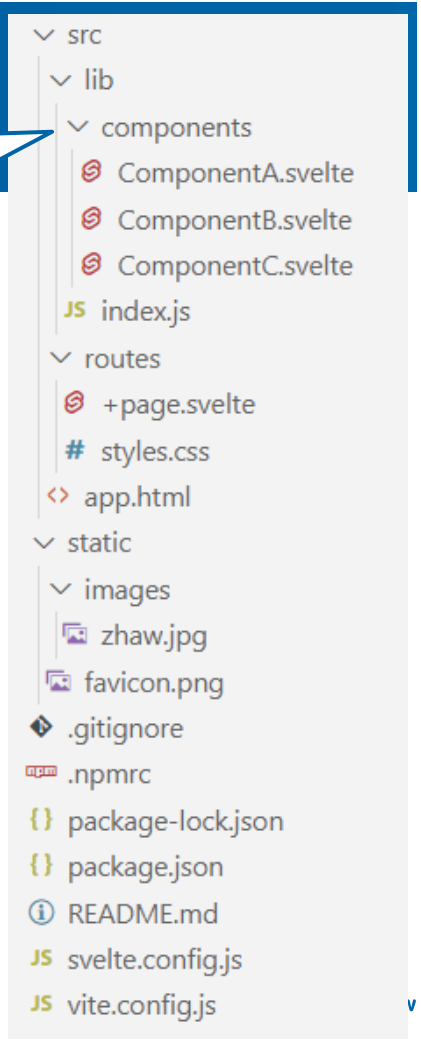
<style>
    p {
        color: red;
    }
</style>
```

This is Component B

ComponentB.svelte

```
<p>This is Component B</p>

<style>
    p {
        color: blue;
        font-style: italic;
        font-weight: bold;
    }
</style>
```



Komponenten importieren und verwenden

Name der Komponente (muss mit einem Grossbuchstaben beginnen)

Komponenten importieren

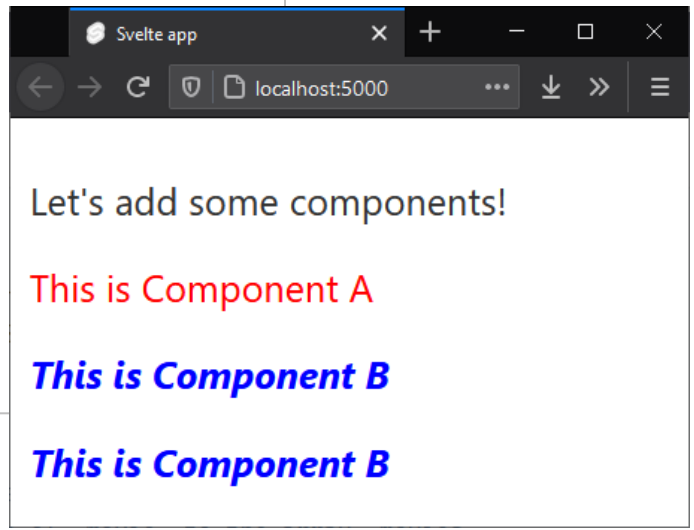
```
<script>
  import ComponentA from "#lib/components/ComponentA.svelte";
  import ComponentB from "#lib/components/ComponentB.svelte";
</script>

<p>Let's add some components!</p>

<ComponentA/>
<ComponentB/>
<ComponentB/>

<style>
</style>
```

Komponenten werden wie HTML-Elemente eingefügt. Der Name des Elements wird definiert durch den im import-Statement definierten Namen. Konvention: Dateiname der Komponente ohne die Endung .svelte.



Daten an Komponenten übergeben

Props

Daten können als *props* an Komponenten übergeben werden (ähnlich wie Funktions-argumente), siehe https://svelte.dev/tutorial/svelte/declaring-props

routes/+page.svelte

```svelte
<script>
    import MessageBox from "#lib/components/MessageBox.svelte";
</script>

<!-- Use MyComponent with the message "Hello World!" -->
<MessageBox message="Hello World!" />

<!-- Use MyCompenent with the default message -->
<MessageBox />
```

Der Wert einer *prop* wird in einem Attribut mit dem gleichen Namen angegeben.

lib/components/MessageBox.svelte

```svelte
<script>
    let { message = "No message provided" } = $props();
</script>

<div class="box">
    <h2>MyComponent</h2>
    <p>Your message: <b>{message}</b></p>
</div>
```

Props müssen als `$props` deklariert werden, damit die Daten von der Parent-Komponente übergeben werden können. Default values (hier «No message provided») sind optional.

MyComponent

Your message: **Hello World!**

MyComponent

Your message: **No message provided**

Objekte als Props

Es können auch komplette **Objekte** als *props* übergeben werden:

routes/+page.svelte

```svelte
<script>
    import ChatMessage from "#lib/components/ChatMessage.svelte";

    let messages = $state([
      {
        text: "Hello PT",
        sender: "Max"
      },
      {
        text: "Hello DM",
        sender: "Alex"
      }
    ])
</script>

{#each messages as message}
  <ChatMessage message={message}/>
{/each}
```

lib/components/ChatMessage.svelte

```svelte
<script>
let { message } = $props();
</script>

<div class="box">
  <h2>Chat Message</h2>
  <p>{message.sender}: <b>{message.text}</b></p>
</div>
```

Chat Message

Max: **Hello PT**

Chat Message

Alex: **Hello DM**

Kurzübung – Props

Öffne folgende App in deinem Browser:
https://svelte.dev/playground/30c1c89e81ba4f0e93cf75dd88739803?version=5.2.7

Vervollständige den Code in *App.svelte* und *Movie.svelte,* so dass die in *App.svelte* definierten Daten in der Komponente *Movie* wie im Screenshot angezeigt werden.

